ESPRIT
École Supérieure Privée d'Ingénierie et de Technologies

Projet AWD - Validation Intermédiaire (Séance 4)

AutismCare

Document Technique

Problématique :
Comment aider l'enfant autiste à communiquer, apprendre et structurer son quotidien, et offrir aux adultes un outil unique pour organiser et suivre l'accompagnement ?

Classe / Groupe : 4ssa13
Équipe :
- Toute l'équipe - Responsable du *schedule-service* (commun)
- Molka Sadkaoui - Responsable du *consultation-service*
- Dorra Serige - Responsable du *course-eval-service*
- Rouem znidi - Responsable du *game-service*
- Fares Ben Ali - Responsable du *event-service*
- Zakaria Boukardougha - Responsable du *communication-service*
- Ahmed laribi - Responsable du *resource-service*

11 novembre 2026

Table des matières

1 Contexte et Problématique **2**
- 1.1 Présentation du projet . . . 2
- 1.2 Problématique . . . 2
- 1.3 Acteurs du système . . . 2

2 Fonctionnalités Principales **3**

3 Architecture Globale et Choix Techniques **4**
- 3.1 Vue d'ensemble . . . 4
- 3.2 Choix Techniques . . . 4

4 Découpage en Microservices **5**

5 Conception des Microservices (Diagrammes de Classes) **7**
- 5.1 Microservice Commun : schedule-service . . . 7
- 5.2 Microservice : consultation-service . . . 7

6 Contrats d'API et Communications **8**
- 6.1 Communications Inter-services . . . 8
- 6.2 Contrats d'API (Principaux Endpoints) . . . 8
  - 6.2.1 Schedule Service (`/api/schedule`) . . . 8
  - 6.2.2 Consultation Service (`/api/consultations`) . . . 9
  - 6.2.3 Course Service (`/api/courses`) . . . 9

7 Gestion de Projet **10**
- 7.1 Organisation du dépôt Git . . . 10
- 7.2 Repartition des taches . . . 10
- 7.3 Planning et Risques . . . 10

Chapitre 1

Contexte et Problématique

1.1 Présentation du projet

AutismCare est une plateforme dédiée à l'accompagnement des enfants atteints de troubles du spectre de l'autisme (TSA). Elle vise à centraliser les outils d'aide à la communication, d'apprentissage, et d'organisation du quotidien.

1.2 Problématique

Comment aider l'enfant autiste à communiquer, apprendre et structurer son quotidien, et offrir aux adultes (parents, éducateurs, spécialistes) un outil unique pour organiser et suivre l'accompagnement ?

1.3 Acteurs du système

- **Enfant** : Utilise l'application pour communiquer (pictogrammes), jouer et suivre son emploi du temps.
- **Parent** : Gère le quotidien, suit les progrès et interagit avec les spécialistes.
- **Éducateur / Spécialiste** : Crée des contenus, gère les disponibilités et les rendez-vous, ajoute des notes de suivi.
- **Administrateur** : Gère la plateforme, les comptes et les ressources globales.

Chapitre 2

Fonctionnalités Principales

La plateforme est divisée en huit grands blocs fonctionnels, correspondant à un microservice ou une brique de la plateforme :

- **Comptes et rôles** : Inscription, connexion (Keycloak), gestion des rôles et droits d'accès.
- **Emploi du temps** : Routine visuelle, rappels, activités faites ou non.
- **Consultations** : Disponibilités des spécialistes, prise de rendez-vous, statuts, notes de suivi.
- **Cours et évaluations** : Cours par catégorie, exercices, résultats, niveau adapté.
- **Jeux éducatifs** : Émotions, mémoire, scores, récompenses symboliques.
- **Communication** : Tableaux de pictogrammes, demandes de l'enfant, historique.
- **Événements** : Ateliers, inscriptions, gestion des places.
- **Ressources** : Fiches, vidéos, recherche par catégorie et niveau.

Chapitre 3

Architecture Globale et Choix Techniques

3.1 Vue d'ensemble

L'architecture repose sur une approche microservices conteneurisés (Docker Compose). Elle est sécurisée par Keycloak (JWT) et utilise une API Gateway pour router les requêtes.

FIGURE 3.1 – Vue d'ensemble de l'architecture microservices d'AutismCare

3.2 Choix Techniques

— **Frontend** : Angular
— **API Gateway** : Spring Cloud Gateway (port 8090)
— **Découverte de services** : Eureka Server (port 8761)
— **Sécurité** : Keycloak (port 8180) avec validation des tokens JWT
— **Communication Synchrone** : OpenFeign (entre microservices Spring Boot)
— **Communication Asynchrone** : RabbitMQ (port 5672) pour les événements métier
— **Bases de données** : PostgreSQL et MongoDB (1 base par microservice)

Chapitre 4

Découpage en Microservices

Chaque microservice est autonome et gère sa propre base de données.

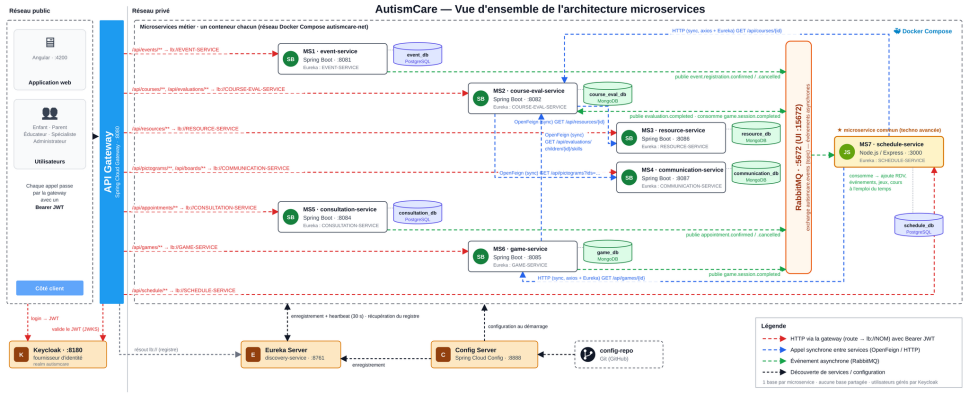


FIGURE 4.1 – Architecture des microservices

Service	Responsable	Responsabilités	Techno / Port	Base
schedule-service (commun)	Toute l'équipe	Emploi du temps visuel, activités, rappels	Node.js : 3000	PostgreSQL
consultation-service	Molka	Spécialistes, rendez-vous, notes de suivi	Spring Boot : 8084	PostgreSQL
course-eval-service	Dora	Cours, exercices, évaluations, compétences	Spring Boot : 8082	MongoDB
game-service	Rouem	Jeux, niveaux, scores, récompenses	Spring Boot : 8085	MongoDB
event-service	Fares	Événements, inscriptions, places	Spring Boot : 8081	PostgreSQL
communication-service	Zakaria	Pictogrammes, tableaux, demandes	Spring Boot : 8087	MongoDB
resource-service	Ahmed	Ressources pédagogiques, médias, recherche	Spring Boot : 8086	MongoDB

TABLE 4.1 – Répartition des microservices et responsabilités

Chapitre 5

Conception des Microservices (Diagrammes de Classes)

5.1 Microservice Commun : schedule-service

Responsable de la présentation : [Nom de l'étudiant]
Ce service gère la planification des activités de l'enfant.

FIGURE 5.1 – Diagramme de classes - Schedule Service

5.2 Microservice : consultation-service

Responsable : Molka
Ce service gère les rendez-vous avec les spécialistes et les notes de suivi.

FIGURE 5.2 – Diagramme de classes - Consultation Service

Chapitre 6

Contrats d'API et Communications

6.1 Communications Inter-services

L'architecture utilise deux modes de communication :

— **OpenFeign (Synchrone)** : Par exemple, le service `course-eval-service` appelle `resource-service` pour récupérer un média (`GET /api/resources/{id}`).

— **RabbitMQ (Asynchrone)** : Par exemple, le `consultation-service` publie l'événement `ConsultationConfirmed`, qui est consommé par le `schedule-service` pour ajouter automatiquement le RDV au planning.

6.2 Contrats d'API (Principaux Endpoints)

6.2.1 Schedule Service (`/api/schedule`)

Méthode	**Endpoint**	**Rôle**	**Codes**
GET	`/{childId}?view=week`	Planning de la semaine	200, 403, 404
POST	`/{childId}/activities`	Ajouter une activité	201, 400, 409
PUT	`/activities/{id}`	Modifier une activité	200, 404, 409
PATCH	`/activities/{id}/status`	Changer le statut (fait/non fait)	200, 400, 404
DELETE	`/activities/{id}`	Supprimer une activité	204, 404

6.2.2 Consultation Service (`/api/consultations`)

Méthode	Endpoint	Rôle	Codes
GET	`/specialists`	Liste des spécialistes	200, 400
POST	`/appointments`	Prendre un rendez-vous	201, 400, 403, 404, 409
GET	`/appointments?childId=`	Historique d'un enfant	200, 403
PATCH	`/appointments/{id}/status`	Confirmer / terminer / annuler	200, 403, 404, 409
POST	`/appointments/{id}/notes`	Ajouter une note de suivi	201, 403, 404

6.2.3 Course Service (`/api/courses`)

Méthode	Endpoint	Rôle	Codes
GET	`?categorie=&niveau=&age=`	Liste filtrée et paginée	200, 400
POST	`/`	Créer un cours	201, 400, 401, 403, 404
POST	`/{id}/contents`	Ajouter un contenu (image, vidéo)	201, 400, 403, 413, 415
POST	`/{id}/assignments`	Affecter le cours à des enfants	201, 400, 403, 404, 409, 422

Chapitre 7

Gestion de Projet

7.1 Organisation du dépôt Git

Le dépôt Git suit la structure imposée par l'enseignant :

```
autismCare-4ssa13/
|-- Backend/              # Code source des microservices (Spring
   Boot, Node.js)
|-- Frontend/             # Application Angular
|-- Documentation/
|   |-- Presentation/     # Diapositives de la seance 4
|   |-- Diagrams/         # Diagrammes UML et d'architecture
|-- README.md
```

7.2 Repartition des taches

- **Molka** : Consultation-service et Integration Keycloak
- **Dora** : Course-eval-service et API Gateway
- **Rouem** : Game-service et Base de donnees MongoDB
- **Fares** : Event-service et RabbitMQ
- **Zakaria** : Communication-service et Frontend Angular
- **Ahmed** : Resource-service et Configuration Eureka
- **Toute l'equipe** : Schedule-service (microservice commun), Documentation

7.3 Planning et Risques

- **Planning** :
  - Semaine 1-2 : Conception UML et contrats d'API.
  - Semaine 3-4 : Developpement des microservices backend.
  - Semaine 5 : Developpement Frontend Angular et integration.
  - Semaine 6 : Tests de communication (Feign/RabbitMQ) et finalisation.
- **Risques principaux** :
  - *Complexite de l'integration RabbitMQ* : Risque de perte de messages. Mitigation : Mise en place de files d'attente persistantes.
  - *Gestion du temps* : Retard sur le frontend. Mitigation : Utilisation de maquettes UI/UX validees tres tot.